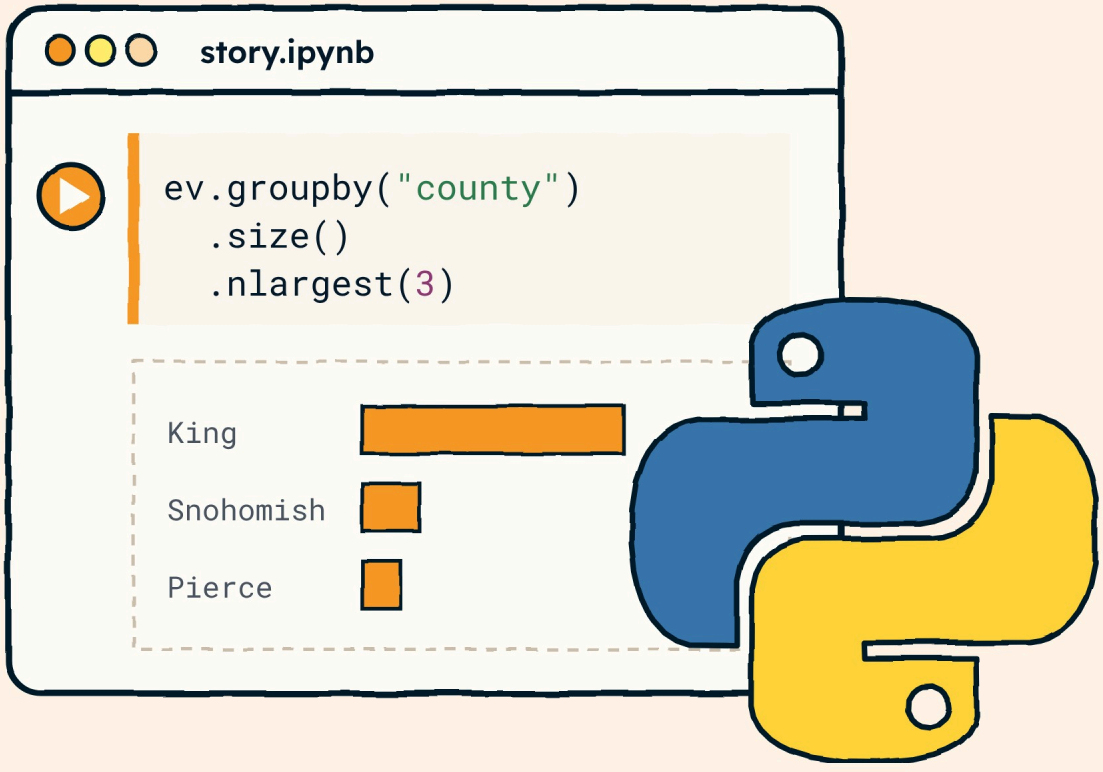
DATA + STORIES SERIES

The Code Behind the Story

Data Skills with Python® for Journalists

Your Toolkit to Analyze and Visualize Data Like a Pro



By: **Amr Eleraqi**

SECOND EDITION

Free sample

This sample of **The Code Behind the Story: Data Skills with Python for Journalists**, second edition, holds the opening of the book, the pages that explain how to use it, and the whole of Chapter 1, “Getting Started with Python and Google Colab,” with its exercises, their solutions and its sources.

The pages are the printed pages of the hardcover and Kindle editions, so the page numbers jump where pages are left out: the contents list shows where everything else is.

The notebooks and data files that go with the book are free at github.com/aeleraqi/Python-for-Journalists.

This sample is free to read and to share as it is, without changes. Copyright © 2026 Amr Eleraqi. All rights reserved. The code examples may be used freely in your own work.

The Code Behind the Story

Data Skills with Python for Journalists

Your Toolkit to Analyze and Visualize Data Like a Pro

Amr Eleraqi

SECOND EDITION

The Code Behind the Story: Data Skills with Python for Journalists
Second edition

Copyright © 2026 by Amr Eleraqi. All rights reserved.

No part of this book may be reproduced in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

Python® and the Python logo are trademarks or registered trademarks of the Python Software Foundation. Google Colab is a trademark of Google LLC. This book is not endorsed by or affiliated with the Python Software Foundation, Google or any other company named in it.

This book is part of the Data + Stories series, which helps professionals turn data into stories.

Every result printed in this book was produced by running its code with the versions of Python and its libraries that Google Colab provided in October 2026, and with the versions current that month of the few libraries that Colab does not include. Websites, services and software change; the boxes marked “As of October 2026” give the date of each fact that may have changed since. The author has checked the facts in this book with care, but cannot accept liability for errors or for the use made of them.

The datasets used in the book belong to their publishers and are used under their licenses, which are named where each dataset first appears.

Code and data: github.com/aeleraqi/Python-for-Journalists

Contents

Introduction vii

Chapter 1

Getting Started with Python and Google Colab 1

1.1 Why a journalist learns Python 1
1.2 What Python is 2
1.3 Your workspace: Google Colab 3
1.4 First steps 6
1.5 Variables and types of data 9
1.6 Numbers and arithmetic 13
1.7 Comparisons and logic 16
1.8 Working with text 18
1.9 Lists, dictionaries, tuples and sets 22
1.10 Decisions and repetition 27
1.11 Functions 31
1.12 Libraries, import and help 32
1.13 Errors you will meet 35
1.14 DataFrames: your first table 37
1.15 Hands-on exercises 40
1.16 Project: building and analyzing a table of articles 44
1.17 Habits worth keeping 49

Chapter 2

Data Collection Techniques 57

2.1 Why data collection matters 57
2.2 Types of data and structures of data 58
2.3 Common file formats 62
2.4 Importing data files 64
2.5 Colab and outside sources 75
2.6 Combining DataFrames 82
2.7 Web scraping 85
2.8 RSS feeds 97
2.9 Extracting the text of articles 101
2.10 Hands-on exercises 103
2.11 Project: a tracker for a regulator's press releases 106
2.12 Habits worth keeping 110

Chapter 3

Data Cleaning and Preparation 119
3.1 Why cleaning comes first 119
3.2 Tools and data for this chapter 122
3.3 A first look at the table 124
3.4 Working with columns 130
3.5 Cleaning text 137
3.6 Missing values 142
3.7 Duplicates 150
3.8 Dates 153
3.9 Hands-on exercises 157
3.10 Project: Washington's electric cars 160
3.11 Habits worth keeping 166

Chapter 4
Exploratory Data Analysis (EDA) 175
4.1 Looking before you write 175
4.2 Data manipulation techniques 180
4.3 Univariate analysis: one column at a time 187
4.4 Bivariate and multivariate analysis 196
4.5 Practical tips for EDA 204
4.6 Case study: does air conditioning go with higher prices? 205
4.7 Bonus: an automated first look with data-profiling 208
4.8 Hands-on exercises 211

Appendix A: pandas cheat sheet 225

Appendix B: Exploring a new dataset 229

About the author 235

Introduction

A new kind of source

The stories we tell are no longer only about who, what, when and where. Readers also want to know why and how, and more and more often the answer sits in data: in government reports, company filings, court records, environmental monitoring, social media and the spreadsheets that sources send us. Data is not only for technical specialists. If you cover politics, health, education, business or culture, it can deepen your reporting, test a claim and make a complicated issue clear.

Data is not always easy to work with, though. Spreadsheets take you a long way and then stop. An Excel sheet holds at most 1,048,576 rows, sifting through thousands of records by hand takes days, and a series of clicks leaves no record that an editor can check. This is where code comes in, not as a replacement for reporting but as a way to do more of it. This book shows how Python, a programming language that is free and easy for beginners to read, can help you find and check the stories in your data.

Learning to code as a journalist

When I wrote my first line of code, `print("Hello, World!")`, it was more than a technical exercise: it was the beginning of a different way of working as a journalist. Code lets you ask questions of data that a spreadsheet cannot answer, and in a profession built on curiosity, it became my way into the stories hidden in rows and columns.

We interview human sources: we listen, follow up on what they say, check it against other sources and notice what they leave out. Code lets us interview data in the same way. We ask a dataset a precise question, look at the answer, ask the next question and compare what we find with other records. A dataset holds a story, but it does not tell it without some coaxing, and like any source it can mislead you if you do not check it.

Coding is not only for “data journalists”. It is for any journalist who wants to work faster and dig deeper. With a few lines of Python you can:

- repeat a dull task, such as collecting or cleaning the same kind of file every week, in seconds;
- analyze tables far too large to read, and find the patterns and the anomalies in them;

- turn numbers into charts that make a complicated finding clear to you and to your readers.

From computer-assisted reporting to Python

Journalists have used computers to analyze records for decades. The early practitioners of computer-assisted reporting (CAR) worked with simple databases and spreadsheets to count, compare and find anomalies that became investigative leads. Code takes the same idea further. Python and its libraries let a reporter collect data from websites, clean it, analyze it and chart it in one place, and keep every step written down so that the work can be checked and repeated.

Take an investigation into campaign contributions. With thousands of donations from people and organizations, a reporter could spend days reading the records. With a few lines of code you can group the donations by donor, by size or by region, and see in minutes which patterns deserve a closer look. The code does not write the story. It shows you where to dig, and it leaves a record of how each number was produced.

What you will learn

The book has four chapters, and each one builds on the one before.

- **Chapter 1** starts from nothing. You open a notebook in Google Colab, in your browser, and learn the Python that the rest of the book uses: values, text, lists and dictionaries, decisions, loops, functions and your first table.
- **Chapter 2** is about collecting data: reading Excel, CSV, text and JSON files, loading data from Google Drive, Kaggle and GitHub, combining tables, and gathering data from the web by scraping pages, reading news feeds and extracting the text of articles.
- **Chapter 3** cleans a real public register of 150,482 electric vehicles: finding what is wrong or unclear in it, fixing it step by step and reporting from the result with the caveats it needs.
- **Chapter 4** explores a file of house prices before you trust any number in it: selecting, grouping and reshaping the table, then summaries, distributions, outliers, relationships between columns and charts, a short case study and an automated first report.

The appendices give a pandas cheat sheet and a checklist for exploring a new dataset, with a plan for your own first project.

Who this book is for

This book is for journalists at any stage: reporters with years of experience and students starting out, people who have never written a line of code and people who have tried and given up. If you have felt overwhelmed by a spreadsheet, or wondered how to find the story in a large set of numbers, it is for you, whatever your beat.

You do not need a background in mathematics, statistics or computer science. Each idea is explained when you first need it, with an example from a newsroom. You need a computer with a web browser, a Google account and the patience to type the code yourself.

How to use this book

Work in Google Colab. Everything in the book runs in Google Colab, a free service that runs Python in your browser, so you do not need to install anything. Chapter 1 shows how to start. Type the code into your own notebook and run it as you read; your fingers learn where the brackets and colons go.

Code and results. Code appears in shaded blocks with an orange line on the left. What the code prints appears right under it in a frame with a dashed border, as it does in Colab, and tables appear as Colab shows them. Every result in the book was produced by running the code with the versions that Colab used in October 2026: Python 3.13, pandas 2.2.3, NumPy 2.1.3, Matplotlib 3.10.0 and seaborn 0.13.2. If your results differ, check your versions first; section 1.12 shows how.

Data and notebooks. The data files and a notebook for each chapter are in the book's repository on GitHub, github.com/aeleraqi/Python-for-Journalists. Each chapter shows how to bring its files into Colab.

Boxes. Several kinds of box recur through the chapters:

- **Tip**: a shortcut or a good habit.
- **Note**: background that helps you understand a result.
- **Common mistakes**: errors that beginners often make, and how to avoid them.
- **As of October 2026**: facts that can change, such as software versions, website rules and terms of use, with the date and source of each.
- **Your turn**: short exercises at the end of most sections. Their solutions are at the end of each chapter.
- **Solution**: the answer to a hands-on exercise, right after it.

- **Check your knowledge**: a short quiz, with the answers.
- **Chapter summary**: the main points of the chapter, at its end.

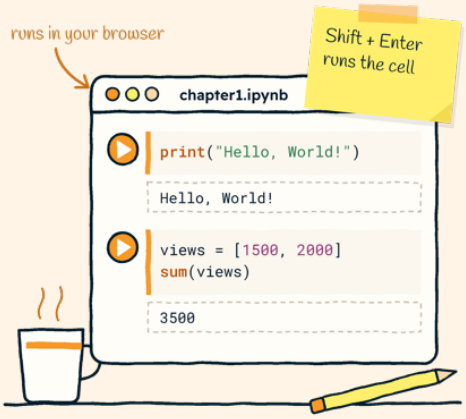
Exercises and projects. Each chapter ends with hands-on exercises, a project that uses the chapter's data for a small piece of reporting (in Chapter 4, a case study), a quiz and a summary. Try each exercise before you read its solution; you will learn more from a wrong attempt than from a right answer you read.

Sources. Facts in the text that come from documentation, data publishers or other references carry a number in square brackets, such as [3]. The full reference is in the list of sources at the end of each chapter.

Chapter 1

Getting Started with Python and Google Colab

Open a notebook in your browser, learn the Python you need for data work and build your first table of articles.



1.1 Why a journalist learns Python

Reporters work with data more often than the job title suggests. Election results, hospital waiting times, court records, budget lines and company filings all arrive as tables, and the story usually sits in a comparison that nobody has made yet. A spreadsheet handles the first few thousand rows well. It struggles when the file is too large to open, when the same cleaning has to be done every week, or when an editor asks how a number was produced and the answer is a series of clicks that nobody wrote down.

Python takes over at that point. A few lines of code can read a file with a million rows, repeat a week's cleaning in seconds and leave behind a record of every step that a colleague can check. You do not need to become a programmer to get there, and you do not need to be a "data journalist". This chapter is written for reporters, editors and freelancers on any beat who want to ask more of the data they already receive.

By the end of the chapter you will have a working notebook in Google Colab, you will know the Python that every later chapter uses (values and variables, text, lists and dictionaries, decisions, loops and functions) and you will have built, filtered and sorted your first table of articles. Most sections end with a short set of exercises; try them before you look at the solutions at the end of the chapter.

1.2 What Python is

Python is a general-purpose programming language. It was created in the early 1990s by Guido van Rossum at the Centrum Wiskunde & Informatica (CWI), a research institute in the Netherlands, as a successor to a teaching language called ABC [1]. Its design favors code that reads almost like plain English, which is one reason it became a common first language in schools and newsrooms. Today the same language runs websites, scientific analyses, automation scripts and machine-learning systems, so the skills you learn here carry over to many other kinds of work.

Why choose Python

Four reasons matter most for journalists (Figure 1.1). The first is readability: a line such as `if views > 1000:` says what it does, which makes code easier to learn and easier for an editor to check. The second is the size of its library collection. Libraries are packages of ready-made tools that other people have written and shared: pandas for working with tables, Beautiful Soup for reading web pages, Matplotlib for charts and NLTK or spaCy for analyzing text. The third is range. The same language collects data from the web, cleans it, analyzes it and draws the chart, so you do not have to move between programs. The fourth is its community, which has produced tutorials, forums and answered questions on almost any problem you will meet. Python has weaknesses too, and Table 1.1 sets them beside its strengths.

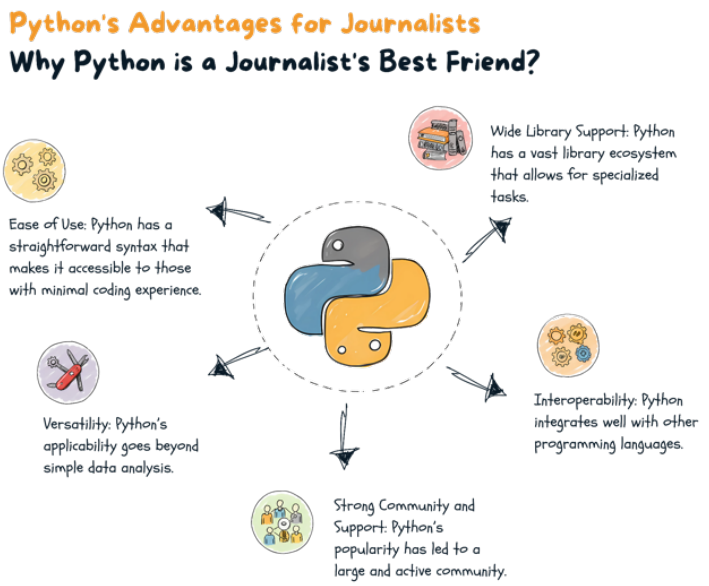


Figure 1.1. Why Python suits journalists: it is easy to read, it has a library for almost every task, it works with other languages and tools, and a large community supports it.

Table 1.1. Python for data journalism: strengths and weaknesses.

Strengths	Weaknesses
One language for scraping, analysis, automation and machine learning	Fewer specialized packages than R for some statistical methods
Readable syntax that beginners pick up quickly	Several charting libraries, each with its own way of writing code
Works alongside other languages and tools, such as SQL databases	Many ways to install and run it, which can confuse a beginner
A large community and a great deal of free teaching material	

Installing Python on your own computer

You do not need to install anything to follow this book. Google Colab, described in the next section, runs Python on Google's computers and shows the results in your browser. If you later want to run Python on your own machine, for example in Jupyter Notebook or VS Code, you can install it from the official website, python.org.

On Windows, python.org now offers the **Python install manager**, which you can also get from the Microsoft Store. After it is installed, the command `python` starts Python, and the command `py` does the same and can also install and manage versions [2]. On macOS, download the installer package from python.org and run it. To check that the installation worked, open the Command Prompt (Windows) or the Terminal (macOS and Linux) and type `python --version`; on a Mac the command is often `python3 --version`. Python comes with pip, a tool that installs extra libraries, and `pip --version` (or `pip3 --version`) confirms that it is there.

As of October 2026

The latest Python release on python.org is Python 3.14.8, released on 30 September 2026 [3]. The python.org documentation says that the traditional Windows installer is deprecated from Python 3.14 and will not be produced for Python 3.16 or later [2]. Download pages and installers change often, so follow the instructions on python.org when they differ from these.

1.3 Your workspace: Google Colab

Google Colab (short for Colaboratory) is a free service from Google that lets you write and run Python in a web browser. Each file is a **notebook**: a page of cells, some holding code and some holding text, with

When a Colab session ends, your variables and uploaded files go with it. The notebook itself stays in Drive.

the results of the code shown under it. Notebooks are saved in your Google Drive, so you can open your work from any computer and share it the way you share a document. Colab runs the code on Google's servers, which means nothing has to be installed on a newsroom laptop that you may not have permission to change.

As of October 2026

Colab's runtime uses Python 3.13 and pandas 2.2.3, and it comes with most of the other libraries this book uses already installed, among them NumPy, Matplotlib, seaborn, Requests and Beautiful Soup [4, 5]. Every result printed in this book was produced with those same versions. Google updates the runtime several times a year; section 1.12 shows how to check which versions your notebook is using.

1.3.1 Opening Colab

You need a Google account. Then:

1. Go to `colab.new` in your browser. The address opens a new, empty notebook.
2. Sign in with your Google account if Colab asks you to.
3. Click the name at the top left, which starts as **Untitled0.ipynb** or similar, and give the notebook a name you will recognize later, such as **chapter1**.

At any later time you can start another notebook from the menu with **File > New notebook in Drive**.

Now write your first program. Click inside the empty code cell, type the two lines below and run the cell by pressing `Shift` + `Enter` or by clicking the round Play button at the left of the cell.

```
message = "Hello, World!"
print(message)
```

```
Hello, World!
```

The first line stores the text `"Hello, World!"` under the name `message`; the second line prints it below the cell. In this book, what a cell prints appears in a frame with a dashed border right under the code, as it does in Colab. The first time you run a cell, Colab takes a few seconds to connect to a computer, so a short wait is normal.

1.3.2 The Colab interface

The menu bar at the top of the notebook holds seven menus. Table 1.2 lists the commands you will use most often while working through this book.

Table 1.2. Colab's menus and the commands a beginner uses most.

Menu	What you will use it for
File	Create a new notebook (**New notebook in Drive**), open or upload one, save a copy in Drive or GitHub, look at earlier versions (**Revision history**), and download the notebook as an `.ipynb` or `.py` file
Edit	Undo and redo, copy and paste cells, **Find and replace**, and **Clear all outputs**
View	Show the **Table of contents** of a long notebook and collapse its sections
Insert	Add a **Code cell** or a **Text cell**, or browse **Code snippets**
Runtime	**Run all** cells, **Interrupt execution** when code takes too long, **Restart session** and **Change runtime type** to ask for more computing power
Tools	Open the **Command palette**, **Settings** and the list of **Keyboard shortcuts**
Help	Read the **Frequently asked questions** and the **release notes**, or report a bug

Below the menus, a notebook is a list of cells of two kinds. A code cell holds Python. You click inside it to edit it and press `Shift` + `Enter` to run it; whatever the code prints, draws or gets wrong appears directly under the cell. Each cell can be run on its own, which lets you build an analysis one small step at a time and check each step before you take the next.

A text cell holds notes written in Markdown, a simple way of formatting text in which `#` at the start of a line makes a heading and `*word*` makes italics. You double-click a text cell to edit it and press `Shift` + `Enter` to see it formatted. Use text cells to say what each part of the notebook does and where the data came from. A colleague who opens your notebook, or you in six months, will need those notes more than you expect.

1.3.3 Useful keyboard shortcuts

Table 1.3 lists the shortcuts that save the most time. On a Mac, press `Cmd` where the table says `Ctrl`. The shortcuts that start with `Ctrl` + `M` take two steps: press `Ctrl` + `M`, release both keys, then press the letter.

Table 1.3. Keyboard shortcuts in Colab.

Shortcut	What it does
`Shift` + `Enter`	Run the cell and move to the next one
`Ctrl` + `Enter`	Run the cell and stay on it
`Alt` + `Enter`	Run the cell and insert a new code cell below it
`Ctrl` + `M`, then `B`	Insert a code cell below
`Ctrl` + `M`, then `M`	Turn the cell into a text cell
`Ctrl` + `M`, then `Y`	Turn the cell into a code cell
`Ctrl` + `M`, then `D`	Delete the cell
`Ctrl` + `M`, then `I`	Interrupt code that is still running
`Ctrl` + `/`	Turn the selected lines into comments, or back into code

Tools > Keyboard shortcuts shows the full list, and lets you change any shortcut you find awkward.

1.3.4 The book's notebooks

The code of this book is also available as Colab notebooks in the book's repository on GitHub, `github.com/aeleraqi/Python-for-Journalists`. Open a notebook there and click the **Open in Colab** badge at its top, and Colab loads a copy you can run and change. Typing the code yourself teaches more, because your fingers learn where the brackets and colons go, but the notebooks help when you want to check a result or catch up after a missed step.

1.4 First steps

1.4.1 A first script: checking a phone number

Reporters collect phone numbers from sources all the time, and numbers copied from emails or business cards often lose their country code. A short script can catch the most common problem. The rule in this example is deliberately simple: a usable international number starts with `+` and is at least 10 characters long.

To build it, open a new notebook and rename it phone_validator. Add a text cell at the top (click **+ Text** in the toolbar) and type `# Checking a source's phone number`, which Colab will show as a heading. Then add a code cell (click **+ Code**) and type:

```
# Ask for a phone number
phone_number = input("Enter a phone number to check: ")

# Check that it starts with a country code and is long enough
if phone_number.startswith("+") and len(phone_number) >= 10:
    print(f"{phone_number} looks valid.")
else:
    print(f"{phone_number} is NOT valid.")
```

```
Enter a phone number to check: +44 20 7946 0958
+44 20 7946 0958 looks valid.
```

When you run the cell, `input()` shows the question and a box under the cell. Type a number, press `Enter`, and the cell prints its verdict; the output above shows what happens when you type `+44 20 7946 0958`. The `if` line asks two things at once: whether the text starts with a plus sign (`startswith("+")`) and whether it has at least 10 characters (`len()` counts them, spaces included). Only when both answers are yes does the first `print()` run; otherwise the line under `else:` runs. The `f` before the quotation marks makes an **f-string**, which puts the value of `phone_number` into the text in place of `{phone_number}`. Section 1.8 explains f-strings, and section 1.10 explains `if` and `else`.

`len()` counts spaces too, so a short number padded with spaces can still pass this test.

Run the cell again and try other numbers. A number typed as `123-456-7890` fails because it has no country code, and `+20 2` fails because it has only five characters. The test is rough on purpose: it does not check that the characters are digits or that the country code exists, and a number can pass it and still be wrong. Code like this narrows down the numbers worth a second look; it does not replace calling the number. Figure 1.2 sums up the path you have just taken, from an empty notebook to a script that works.



Figure 1.2. From an empty notebook to a working script.

1.4.2 Words you will keep seeing

Programming has its own vocabulary, and a few words come up in almost every line of this book. Table 1.4 defines them with an example of each.

Table 1.4. Words you will keep seeing.

Term	Meaning	Example
Function	A named command that does one job. You run it by writing its name followed by parentheses	`len("hi")`
Argument	The value you put inside the parentheses	in `len("hi")`, the argument is `"hi"`
Return value	What the function hands back. You can store it in a variable	`n = len("hi")` stores 2 in `n`
Method	A function that belongs to a type of value, written after a dot	`"hi".upper()`
Object	Any value: a number, a piece of text, a list. Every object has a type	`8` is an `int`
Mutable, immutable	Whether a value can be changed after it is created	a list can be changed; a string cannot
Built-in	Comes with Python, with no `import` needed	`print`, `len`, `type`

The difference between a function and a method trips up many beginners. A function stands on its own and takes the value in its parentheses, as in `len(headline)`. A method is attached to a value with a dot, as in `headline.upper()`, and works on that value.

1.4.3 What a cell shows

A code cell can run many lines, but it shows on its own only the value of its last line. Anything before that is shown only if you print it.

```
views = 1500
comments = 275
views
comments
```

```
275
```

Python ran all four lines, but only the value of `comments`, on the last line, appears below the cell. To see both, print them, or ask for both on the last line, separated by a comma:

```
print(views)
views, comments
```

```
1500
```

```
(1500, 275)
```

The pair in parentheses is a **tuple**, a fixed group of values, which section 1.9 explains.

1.4.4 Comments

A comment is a note for human readers, which Python ignores when it runs the code. A comment starts with `#` and runs to the end of the line. It can sit on a line of its own or after the code it explains:

```
# Views on the day the article was published
views = 1500
comments = 275  # comments were counted the next morning
```

For a longer note, start each line with `#`. You may see older code that wraps a block of notes in triple quotation marks (`"""`). That is a string rather than a comment, and Python uses such strings for a specific purpose, describing what a function does (section 1.11), so it is clearer to keep to `#`. In Colab you can select several lines and press `Ctrl` + `/` to comment them out, which is a quick way to switch off part of the code while you test the rest.

Good comments explain why a line is there, not what it does: `# exclude test accounts created by the data provider` helps a reader, while `# filter the rows` repeats the code. When you change the code, change its comments too.

1.5 Variables and types of data

1.5.1 Variables

A **variable** is a name that refers to a value (Figure 1.3). You create one with the equals sign, the **assignment operator**: the name goes on the left and the value on the right. Read `=` as “store”, not as “equals”: Python first works out the right side, then stores the result under the name on the left.

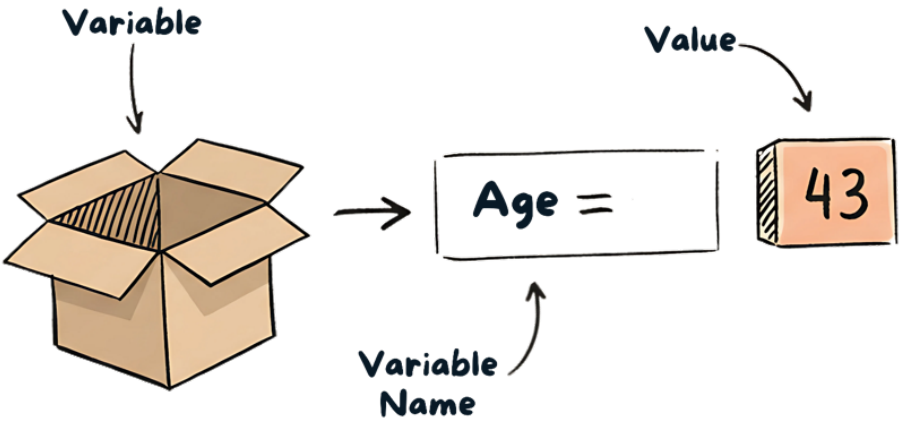


Figure 1.3. A variable is a labeled box: the name on the label, the value inside.

```
age = 43
age
```

```
43
```

A variable holds the last value you gave it. That lets you update a value in place, which is how a counter works:

```
count = 0
count = count + 1    # take count, add 1, store it back in count
count += 1           # the same thing, written shorter
count
```

```
2
```

`+=` has siblings for the other operations: `-=`, `*=` and `/=`.

1.5.2 Types of data

Every value in Python has a type, and four types cover most of what a journalist stores (Figure 1.4):

- An **integer** (`int`) is a whole number, positive or negative, such as a count of articles or votes.
- A **float** is a number with a decimal point, such as an average rating or a percentage.
- A **string** (`str`) is text, written between quotation marks: a name, a headline, a description.
- A **Boolean** (`bool`) is either `True` or `False`, and records the answer to a yes-or-no question.

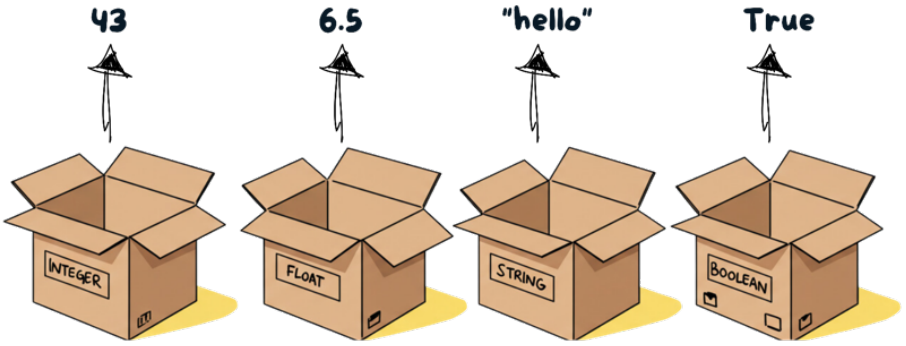


Figure 1.4. Four basic types: an integer, a float, a string and a Boolean.

```
number_of_articles = 10                                    # an integer
average_rating = 4.7                                       # a float
headline = "Breaking News: Election Results Announced"     # a string
is_published = True                                        # a Boolean

print(type(number_of_articles), type(average_rating))
print(type(headline), type(is_published))
```

```
<class 'int'> <class 'float'>
<class 'str'> <class 'bool'>
```

The function `type()` reports the type of any value, which is the first thing to check when a calculation behaves strangely. Text in quotation marks is always a string, even when it looks like a number: `"1500"` is four characters, not a number. `print()` accepts several values at once, separated by commas, and prints them on one line with a space between each:

Two numbers stored as text do not add up: `+` glues them together, with no error to warn you.

```
headline = "Climate Change Summit Yields New Agreements"
word_count = 850
average_reading_time = 3.5

print("Headline:", headline, "Word count:", word_count,
      "Reading time (minutes):", average_reading_time,
      "Published:", is_published)
```

```
Headline: Climate Change Summit Yields New Agreements Word count: 850 Reading time
(minutes): 3.5 Published: True
```

The line is long, so we broke it after a comma; Python allows a line break anywhere inside parentheses.

> **Tip**
>
> **Naming variables.** Choose names that say what the value is: `headline`, `view_count` and `is_published` explain themselves, while `a`, `x` and `data1` tell a reader nothing. For names of more than one word, use lowercase words joined by underscores (`total_views`, `author_name`), a style called **snake_case**. A name must start with a letter or an underscore, never a digit, and cannot contain spaces, so `2nd_place` and `my name` are not allowed. Python's **keywords**, such as `if`, `for`, `class`, `True` and `import`, cannot be used as names at all. Names of built-in functions, such as `print`, `len`, `sum` and `type`, can be reused, but doing so breaks the function for the rest of the session, so avoid them too.

1.5.3 Converting between types

Data rarely arrives in the type you need. Numbers scraped from a web page or typed into a form are usually text, and you have to turn them into numbers before you can add them up. Each type has a function with its own name that converts a value into that type: `int()`, `float()`, `str()` and `bool()`.

```
print(int("25") + 5)    # text to a whole number
print(float("3.5") * 2) # text to a decimal number
print(str(12742))       # a number to text
print(int(7.6))         # cuts off the decimals; it does not round
```

```
30
7.0
12742
7
```

Notice the last line: `int(7.6)` gives 7, not 8, because `int()` drops everything after the decimal point. Use `round()` when you want to round. Conversions fail loudly when they cannot work. Text that is not a number cannot become one:

```
int("1,500")
```

```
ValueError: invalid literal for int() with base 10: '1,500'
```

The comma makes `"1,500"` something other than a plain number, so `int()` stops with a `ValueError`. Chapter 3 shows how to remove such characters from a whole column at once. Mixing text and numbers in one calculation fails too:

```
"Views: " + 1500
```

```
TypeError: can only concatenate str (not "int") to str
```

Python will not guess whether you meant to do arithmetic or to join text, so it stops with a `TypeError`. Convert the number first, `"Views: " + str(1500)`, or use an f-string as section 1.8 shows.

Your turn

1. Create `price = 20`, add 5 to it with `+=` and print the price and its type.
2. Create three variables, `full_name`, `age` and `city`, and print them with one `print()` call.
3. These names are not allowed: `2nd_place`, `my name`, `class`. Write a valid version of each and give it a value.
4. `answer = "42"` is text. Turn it into a number, add 8 and print the result.
5. Print `9.99` as a whole number with `int()`. What happened to the `.99`?

1.6 Numbers and arithmetic

Python works as a calculator. Table 1.5 shows its arithmetic operators, with the result of each example as Python prints it, and Figure 1.5 shows them at a glance.

Table 1.5. Python's arithmetic operators.

Operator	Operation	Example	Result
`+`	Addition	`10 + 5`	`15`
`-`	Subtraction	`20 - 5`	`15`
`*`	Multiplication	`4 * 5`	`20`
`/`	Division	`10 / 2`	`5.0`
`**`	Exponent (power)	`2 ** 3`	`8`
`%`	Remainder after division	`10 % 3`	`1`
`//`	Division rounded down to a whole number	`9 // 2`	`4`

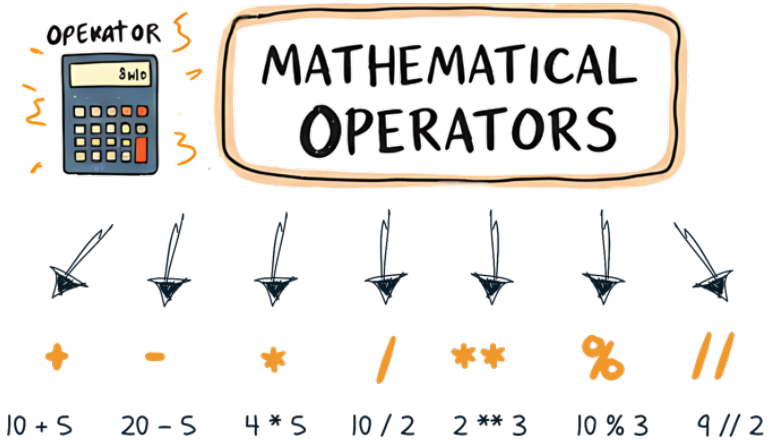


Figure 1.5. Python's arithmetic operators.

Two details save confusion later. Division with `/` always returns a float, even when it comes out even: `10 / 2` is `5.0`, not `5`. And any calculation that involves a float gives a float. Python also follows the usual order of operations, so multiplication and division happen before addition and subtraction unless parentheses say otherwise:

```
print(3 + 2 * 4)      # 2 * 4 first, then + 3
print((3 + 2) * 4)    # the parentheses come first
```

```
11
20
```

The remainder operator `%` answers questions such as "is this number even?" (the remainder after dividing by 2 is 0), and `//` with `%` together split a quantity into whole units and what is left over:

```
minutes = 100
print(minutes // 60, "hours and", minutes % 60, "minutes")
```

```
1 hours and 40 minutes
```

Computers store decimal numbers in binary, and some fractions cannot be stored exactly, so a tiny error sometimes appears in the last decimal place:

```
7.6 - 8
```

```
-0.40000000000000036
```

Never test a result like this with ==. Round it first, or use `math.isclose()` to check it is close enough.

The answer should be `-0.4`. The error is far too small to matter in journalism, but it explains the odd-looking digits you will sometimes see; `round()` tidies them away. Two more built-in functions and one library help with arithmetic: `abs()` returns a number without its sign, `round()` rounds to the number of decimal places you ask for, and the `math` library, which comes with Python, has functions such as `math.sqrt()` for square roots.

```
import math

print(abs(-4))
print(round(3.14159, 2))
print(math.sqrt(36))
```

```
4
3.14
6.0
```

Example: measuring engagement

Your editor wants a quick picture of how three recent articles performed. Together they drew 1,500 views and 275 comments. Find the total engagement (views plus comments), the views you would expect next month if they doubled, the average engagement per article and the views per article if each article drew the same share.

```
views = 1500
articles = 3
comments = 275

total_engagement = views + comments
print("Total engagement:", total_engagement)

projected_views = views * 2
print("Projected views next month:", projected_views)

engagement_per_article = (views + comments) / articles
print("Average engagement per article:", engagement_per_article)

views_per_article = views / articles
print("Views per article:", views_per_article)
```

```
Total engagement: 1775
Projected views next month: 3000
Average engagement per article: 591.6666666666666
Views per article: 500.0
```

The average engagement prints with many decimal places, because 1,775 divided by 3 does not come out even. `round()` keeps as many places as you ask for:

```
round(engagement_per_article, 2)
```

```
591.67
```

Your turn

1. A press room is 7.5 meters by 4 meters. Print its area, then the area rounded to one decimal place.
2. How many seconds are there in one day? Print the number.
3. Is 2026 an even year? Use `%` and print `True` or `False`.
4. Print the average of 18, 25 and 31, rounded to two decimal places.
5. Two temperatures were recorded: -4 and 11. Print the difference between them as a positive number.

1.7 Comparisons and logic

Comparison operators compare two values and answer `True` or `False` (Table 1.6). They are the building blocks of every filter: “views greater than 1,000”, “country equal to Egypt”, “year not equal to 2020”.

Table 1.6. Comparison operators.

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`>`	Greater than	`5 > 3`	`True`
`<`	Less than	`5 < 3`	`False`
`>=`	Greater than or equal to	`5 >= 5`	`True`
`<=`	Less than or equal to	`5 <= 4`	`False`

Be careful to tell `==` from `=`. A single equals sign stores a value in a variable; a double one asks whether two values are equal. Comparisons work on text too, and there they are exact: `"AI" == "ai"` is `False`, because a capital letter is a different character from a lowercase one.

Logical operators combine comparisons (Table 1.7). `and` is true only when both sides are true, `or` is true when at least one side is, and `not` turns true into false and false into true. The examples assume an article with 1,500 views published in 2024.

Table 1.7. Logical operators.

Operator	True when	Example	Result
`and`	both conditions are true	`(views > 1000) and (year == 2024)`	`True`
`or`	at least one condition is true	`(views > 2000) or (year == 2024)`	`True`
`not`	the condition is false	`not (views > 2000)`	`True`

Behind the scenes, Python treats `True` as 1 and `False` as 0. That makes it easy to count how many items pass a test, as later chapters do with whole columns of data:

```
print(True + True + False)
print(sum([True, False, True, True]))
```

```
2
3
```

Example: finding high-performing articles

An article by Jane Doe was published in 2024 and has 1,500 views. Is it a popular article (more than 1,000 views) from 2024? Is it either highly viewed (more than 2,000 views) or published? Does it have 2,000 views or fewer?

```
views = 1500
year = 2024
author = "Jane Doe"
is_published = True

if (views > 1000) and (year == 2024):
    print("A popular article from 2024.")

if (views > 2000) or is_published:
    print("Highly viewed, or published, or both.")

if not (views > 2000):
    print("2,000 views or fewer.")
```

```
A popular article from 2024.
Highly viewed, or published, or both.
2,000 views or fewer.
```

All three conditions are true, so all three messages are printed. Change `views` to 2500 and run the cell again: the first two messages still appear, and the third does not.

Your turn

1. With `age = 20`, write one expression that is `True` only when age is 20 or more and less than 30.
2. `password = "secret12"`. Print `True` if it is at least eight characters long.
3. `headline = "Nile water levels drop"`. Print `True` if the word `"water"` appears in it.
4. An article drew 900 views yesterday and 1,200 today. Print the difference and the total, and print whether views went up or down.

1.8 Working with text

Much of a journalist's data is text: names, headlines, addresses, quotes, categories. A **string** is text between quotation marks. Single and double quotation marks work the same way; use one kind when the text contains the other:

```
quote = "The minister's office declined to comment."
print(quote)
print(len(quote))
```

```
The minister's office declined to comment.
42
```

`len()` counts every character, spaces and punctuation included. Inside a string, a backslash starts an **escape sequence**: `\n` starts a new line and `\t` inserts a tab. They show up as themselves when Colab displays a string, and take effect when you print it:

```
lead = "First paragraph.\nSecond paragraph."
print(lead)
```

```
First paragraph.
Second paragraph.
```

1.8.1 Searching and joining

`in` asks whether one piece of text appears inside another, and `not in` asks the opposite. Both answer `True` or `False`, and both care about capital letters. `+` joins strings, with no space added for you, and `*` repeats them:

```
headline = "Parliament passes new budget"
print("budget" in headline)
print("Budget" in headline)
print("tax" not in headline)
print("Amr" + " " + "Eleraqi")
print("-" * 20)
```

```
True
False
True
Amr Eleraqi
--------------------
```

1.8.2 String methods

Strings come with methods that clean and search text, which you will use constantly once data arrives from the real world. Each one returns a new string (or another result) and leaves the original as it was (Table 1.8).

Table 1.8. Common string methods.

Method	What it does	Returns
`.lower()`, `.upper()`	Changes the case of every letter	a string
`.strip()`	Removes spaces (and line breaks) from both ends	a string
`.replace(old, new)`	Replaces every `old` with `new`	a string
`.split(sep)`	Cuts the text into pieces at `sep`, or at spaces if you give no `sep`	a list
`.count(sub)`	Counts how often `sub` appears	an integer
`.find(sub)`	Gives the position of the first `sub`, or `-1` if it is not there	an integer
`.startswith(sub)`	Says whether the text starts with `sub`	a Boolean

```
email = "   Amr.Eleraqi@Example.COM  "
print(email.strip().lower())

statement = "The minister said the budget was fair. The budget was not fair."
print(statement.count("budget"))
```

```
print(statement.find("budget"))
print(statement.find("tax"))
print(statement.replace("budget", "plan"))
print("Cairo, Giza, Alexandria".split(", "))
```

```
amr.eleraqi@example.com
2
22
-1
The minister said the plan was fair. The plan was not fair.
['Cairo', 'Giza', 'Alexandria']
```

The first line chains two methods: `.strip()` returns a cleaned string, and `.lower()` works on that result. The word “budget” appears twice, and its first appearance starts at position 22, counting from 0; the `-1` for “tax” means the word is not in the statement.

Strings are **immutable**: once created, they cannot be changed in place. That is why every method returns a new string, and why you store the result if you want to keep it, as in `email = email.strip()`.

1.8.3 Indexing and slicing

Each character of a string has a position, its **index**, counting from 0. Negative numbers count from the end, so `-1` is the last character. A **slice** takes a run of characters with `[start:stop]`: the start is included and the stop is not. Leave out the start to begin at the beginning, leave out the stop to run to the end and add a third number for a step.

```
date = "2026-09-20"
print(date[0])      # the first character
print(date[-1])     # the last character
print(date[:4])     # the year: positions 0 to 3
print(date[5:7])    # the month: positions 5 and 6
print(date[8:])     # the day: from position 8 to the end
print(date[::-1])   # the whole string, backwards
```

```
2
0
2026
09
20
02-90-6202
```

Slicing is a quick way to pull a year, a code or an area code out of text that always has the same layout. Asking for a position that does not exist stops with an `IndexError`, and trying to change a character in place fails, because strings are immutable:

```
date[0] = "1"
```

```
TypeError: 'str' object does not support item assignment
```

1.8.4 Putting values into text

Stories need sentences with numbers in them. An **f-string**, a string with `f` before the opening quotation mark, replaces whatever is in curly braces, a variable or a calculation, with its value. After it you can add a colon and a format: `:,` adds thousands separators, and `:.1f` shows one decimal place.

```
city = "Toronto"
population = 2794356
share = 0.4567

print(f"{city} has a population of {population:,} people.")
print(f"The share rose to {share * 100:.1f}%.")
```

```
Toronto has a population of 2,794,356 people.
The share rose to 45.7%.
```

A format changes only what is printed. The variable keeps every decimal, so round once, at the end.

You will also meet the older `.format()` method, which fills each pair of empty braces with the values in order: `"{} has {} people".format(city, population)`. It works the same way, and f-strings are shorter to write and easier to read. `print()` itself takes two useful options: `sep=` sets what goes between the values (a space unless you say otherwise) and `end=` sets what goes at the end (a new line unless you say otherwise).

```
print("data", "water", "nile", sep=" | ")
print("Loading...", end=" ")
print("done")
```

```
data | water | nile
Loading... done
```

Your turn

1. `email = "  Amr.Eleraqi@Example.COM  "`. Print it with no spaces at the ends and in lowercase.
2. Split `"data journalism is fun"` into words and print how many words there are.

3. In `"the dam and the river and the data"`, print how many times the word `"the"` appears.
4. `word = "journalism"`. Print the first four characters, the last three and the word reversed.
5. From `phone = "416-555-0199"`, print the first three digits and the last four.
6. `city = "Cairo"` and `population = 10230350`. Print `"Cairo has 10,230,350 people."` with an f-string.

1.9 Lists, dictionaries, tuples and sets

A single variable holds a single value. Journalism rarely deals in single values, so Python offers several ways of keeping many values under one name. Two of them, lists and dictionaries, will appear on almost every page of this book; tuples and sets turn up less often but are worth knowing.

1.9.1 Lists

A **list** keeps items in order, like the day's headlines in the order they were published. You create one with square brackets and commas between the items, and you add to it with `.append()` (one item) or `.extend()` (several items):

```
headlines = ["Climate Change Report", "Election Results"]

headlines.append("New Policy Updates")
headlines.extend(["Economy News", "Sports Highlights"])
headlines
```

```
['Climate Change Report',
 'Election Results',
 'New Policy Updates',
 'Economy News',
 'Sports Highlights']
```

Colab shows a long list one item per line. Lists use the same indexing and slicing as strings: Python counts from 0 at the front and from -1 at the back (Figure 1.6).

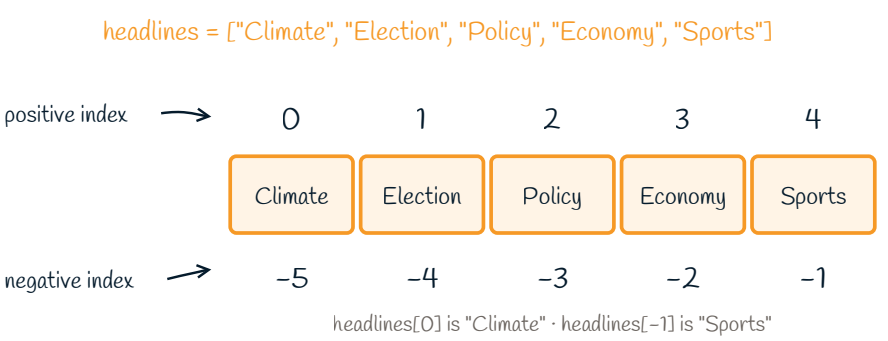


Figure 1.6. Positions in a list. Python counts from 0 at the front and from -1 at the back.

```
print(headlines[0])
print(headlines[-1])
print(headlines[1:3])
```

```
Climate Change Report
Sports Highlights
['Election Results', 'New Policy Updates']
```

Unlike strings, lists are **mutable**: you can change, add and remove items in place. Table 1.9 lists the methods you will use most.

Table 1.9. Common list methods.

Method	What it does
`.append(item)`	Adds one item at the end
`.extend([items])`	Adds several items at the end
`.insert(position, item)`	Inserts an item at a position, moving the rest along
`.remove(item)`	Removes the first copy of an item
`.pop()`	Removes the last item and returns it
`.sort()`	Sorts the list in place; `.sort(reverse=True)` sorts from largest to smallest
`.count(item)`	Counts how often an item appears

```
scores = [70, 95, 88]
scores.append(60)
scores.remove(70)
scores[0] = 100        # replace the first item
scores.sort()
scores
```

```
[60, 88, 100]
```

> ⚠ **Common mistakes**
>
> Methods that change a list, such as `.append()` and `.sort()`, change it in place and return nothing. Writing `scores = scores.append(60)` therefore replaces your list with `None`, Python's way of saying "nothing here". Call such methods on a line of their own. The functions `sorted()` and `len()`, by contrast, leave the list alone and return a result you can store.

A list can hold any mix of values, including other lists. To reach inside a **nested** list, add one pair of square brackets for each layer, reading from left to right:

```
table = [[1, 2, 3], [4, 5, 6]]
print(table[1])       # the second row
print(table[1][2])    # the third value of the second row
```

```
[4, 5, 6]
6
```

1.9.2 Dictionaries

A **dictionary** stores values under labels called **keys**, so you look up a value by its name rather than by its position, the way you look up a word in a dictionary. You write one with curly braces, a colon between each key and its value, and commas between the pairs (Figure 1.7). Keys are unique within a dictionary.

```
article = {
    "title": "Climate Change Report",
    "author": "Jane Doe",
    "views": 1500
}
```



Figure 1.7. A dictionary maps each key to a value.

To add a pair, or to change the value of a key that exists, assign to the key:

```
article["date"] = "2024-01-01"   # a new key
article["views"] = 1600          # a new value for an existing key
article
```

```
{'title': 'Climate Change Report',
 'author': 'Jane Doe',
 'views': 1600,
 'date': '2024-01-01'}
```

You read a value with the key in square brackets, or with `.get()`, which can also take a second value to return when the key is missing:

```
print(article["title"])
print(article.get("author"))
print(article.get("editor"))
print(article.get("editor", "not recorded"))
```

```
Climate Change Report
Jane Doe
None
not recorded
```

The difference shows up when a key is missing. `article["editor"]` would stop the code with a `KeyError`, while `article.get("editor")` returns `None` and lets the code carry on. Use `.get()` when a missing key is normal, and square brackets when a missing key means something has gone wrong and you want to know. Three more methods list the contents of a dictionary: `.keys()`, `.values()` and `.items()`, the last of which gives the key-and-value pairs.

```
print(list(article.keys()))
print(list(article.values()))
```

```
['title', 'author', 'views', 'date']
['Climate Change Report', 'Jane Doe', 1600, '2024-01-01']
```

Dictionaries can hold lists and other dictionaries, which is exactly the shape of the data that most websites and APIs send (Chapter 2). You reach inside one layer at a time, using a key for a dictionary and an index for a list:

```
story = {"title": "Water prices", "tags": ["economy", "Nile"],
         "author": {"name": "Sara Ali", "desk": "Business"}}
print(story["tags"][1])
print(story["author"]["desk"])
```

```
Nile
Business
```

1.9.3 Tuples and sets

A **tuple** is like a list that cannot be changed: you write it with parentheses, and once it exists you cannot add, remove or replace its items. Tuples suit values that belong together and should stay together, such as a pair of coordinates. You can **unpack** a tuple into separate variables in one line:

```
location = (30.0444, 31.2357)   # latitude and longitude of Cairo
latitude, longitude = location
print(latitude)
print(longitude)
```

```
30.0444
31.2357
```

A **set** is a collection with no duplicates and no order. Turning a list into a set is the quickest way to find its distinct values, which is a frequent first question about a column of data:

```
tags = ["data", "water", "data", "Nile", "water"]
unique_tags = set(tags)
print(len(unique_tags))
print(sorted(unique_tags))
```

```
3
['Nile', 'data', 'water']
```



Capitals sort before lowercase letters, so `Nile` comes first. A tag typed `nile` would count as a separate tag.

A set has no positions, so it cannot be indexed, and its items may come out in any order; `sorted()` turns it into a sorted list when you need a stable order. Table 1.10 compares the four structures side by side.

Table 1.10. Four ways to hold many values.

Structure	Written as	Ordered	Can change	Read by
List	`[1, 2, 3]`	Yes	Yes	position, `my_list[0]`
Dictionary	`{"a": 1}`	In the order keys were added	Yes	key, `my_dict["a"]`
Tuple	`(1, 2, 3)`	Yes	No	position, `my_tuple[0]`
Set	`{1, 2, 3}`	No	Yes (add and remove)	not indexed

Your turn

1. `scores = [70, 95, 88]`. Add 60 to the end, remove 70, sort the list and print it.
2. Make a list of five cities and print how many there are and the third one. Then insert `"Alexandria"` at position 1 and print the list.
3. `tags = ["data", "water", "data", "nile"]`. Print how many times `"data"` appears, and how many different tags there are.
4. Build a dictionary `article` with the keys `title`, `year` and `tags` (a list of two tags). Print the year and the second tag.
5. Add a key `author` to that dictionary, change the year to 2027 and print it.
6. `data = [1, 2, [3, 4, ["deep"]]]`. Print the word `"deep"` using indexing.

1.10 Decisions and repetition

1.10.1 Indentation

Python uses indentation, the space at the start of a line, to group lines that belong together (Figure 1.8). Many languages use curly braces `{}` for this; Python uses the indentation itself, much as an outline uses indented sub-points under a main point.

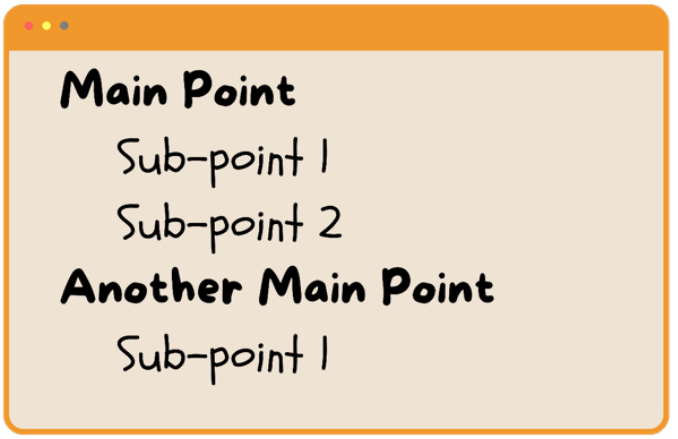


Figure 1.8. Indentation works like an outline: indented lines belong to the line above them.

Python accepts any amount of indentation, as long as every line of a block is indented by the same amount. By convention, though, each level is four spaces: the official style guide for Python code, known as PEP 8, says to “use 4 spaces per indentation level” and that “spaces are the preferred indentation method” [6]. The code in this book follows that convention. When you press `Enter` after a line that ends in a colon, Colab in-

dents the next line for you. If the indentation is missing, Python refuses to run the code at all:

```
if views > 1000:
print("This article is popular.")
```

```
IndentationError: expected an indented block after 'if' statement on line 1
```

An `IndentationError` points to the line Python expected to be indented. The same style guide notes that "Python disallows mixing tabs and spaces for indentation" [6], which is a good reason to let Colab do the indenting.

1.10.2 Making decisions with if, elif and else

An `if` statement runs the lines indented under it only when its condition is true. The line ends with a colon, and an `else:` line can follow it, with the lines to run otherwise:

```
views = 1500

if views > 1000:
    print("This article is popular.")
else:
    print("This article is less popular.")
```

```
This article is popular.
```

When there are more than two possibilities, `elif`, short for "else if", adds conditions that are checked one after another, until one is true:

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)
```

```
C
```

Python checks the conditions from the top and stops at the first one that is true, so the order matters: 78 is not 90 or more, not 80 or more, but is 70 or more, so the grade is C.

1.10.3 Repeating with loops

Much of the work in a newsroom is repetition with judgment: go through every headline of the day and flag the ones about the election, or go through every article and mark those with more than 1,000 views. In Python, a **loop** does the going through, and a condition does the judging (Figure 1.9).

A `for` loop repeats the indented lines under it once for each item of a list. Each time round, the name after `for` (here `headline`) refers to the next item. Put an `if` inside the loop and the test is applied to every item:

```
headlines = ["Climate Change Report", "Election Results", "New Policy Updates"]

for headline in headlines:
    if "Election" in headline:
        print("Election-related headline:", headline)
```

```
Election-related headline: Election Results
```

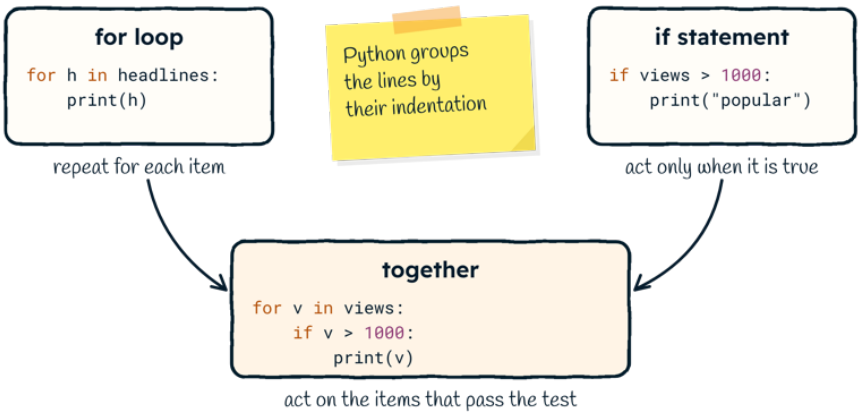


Figure 1.9. Loops repeat an action for each item; conditions decide whether to act.

The test `"Election" in headline` is case-sensitive, so a headline that writes “election” in lowercase would be missed; testing `"election" in headline.lower()` avoids that. To repeat something a set number of times, loop over `range()`. `range(5)` produces the numbers 0 to 4, and `range(2, 11, 2)` the even numbers from 2 to 10, because the stop value is never included:

The Code Behind the Story

```
for i in range(1, 4):
    print("7 x", i, "=", 7 * i)
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
```

A loop can also build up a result as it goes, here a total:

```
daily_views = [1500, 1200, 800, 2000, 950]
total = 0
for v in daily_views:
    total += v
print("Total:", total)
print("Average:", total / len(daily_views))
```

```
Total: 6450
Average: 1290.0
```

A `while` loop repeats as long as its condition stays true. It is less common in data work, because most loops go through a list, and it needs care: if the condition never becomes false, the loop never ends, and you have to stop it with **Runtime > Interrupt execution**.

```
count = 1
while count <= 3:
    print("Attempt", count)
    count += 1
```

```
Attempt 1
Attempt 2
Attempt 3
```



Delete `count += 1` and this loop never stops: `count` stays at 1 forever.

Loops and conditions are the logic behind every filter you will apply. When you work with DataFrames, pandas usually runs the loop for you, as section 1.15 shows, but the idea underneath is the same.

Your turn

1. Write a grade checker for `score = 85`: 90 or more is A, 80 to 89 is B, 70 to 79 is C, anything lower is F.
2. Use `range()` and a `for` loop to print the even numbers from 2 to 10.

3. `marks = [55, 78, 91, 64, 83]`. Loop over the list and print only the marks above 70.
4. Add up the numbers in `marks` with a loop, and print the total and the average.
5. Make a list `headlines` of four headlines of your own and print how many of them mention `"water"`, whatever the capitalization.

1.11 Functions

A **function** is a named block of code that you can run again and again. You have used Python's built-in functions (`print()`, `len()`, `round()`); now you will write your own. A function has two parts: the **definition**, which starts with `def` and says what the function does, and the **call**, which runs it. The names in the definition's parentheses are **parameters**, empty slots that are filled with the **arguments** you pass when you call the function.

```
def engagement_rate(likes, comments, views):
    """Likes plus comments, as a percentage of views."""
    return (likes + comments) / views * 100

rate = engagement_rate(200, 50, 1500)
print(round(rate, 1))
```

```
16.7
```

Running a definition on its own prints nothing; it only saves the function under its name. The string in triple quotation marks at the start of the body is a **docstring**, which describes the function and appears when you ask for help on it (section 1.12). The `return` line hands a value back to whoever called the function, so the result can be stored, printed or used in another calculation.

The difference between `return` and `print()` is the most important thing to learn about functions. `print()` shows a value on the screen and keeps nothing; `return` hands the value back so you can use it:

```
def square_print(n):
    print(n ** 2)

def square_return(n):
    return n ** 2
```

```
result1 = square_print(4)     # shows 16, but keeps nothing
result2 = square_return(4)    # keeps 16
print(result1, result2)
```

```
16
None 16
```

The first function printed 16, but `result1` holds `None`; only `result2` kept the number. A parameter can have a default value, which is used when the caller leaves the argument out:

```
def greet(name="friend"):
    return "Hello, " + name

print(greet())
print(greet("Sara"))
```

```
Hello, friend
Hello, Sara
```

Functions pay off as soon as you do the same calculation twice. If a newsroom defines its engagement rate in one function, every analysis that calls it uses the same definition, and a change to the definition reaches all of them at once.

Your turn

1. Write a function `area(width, height)` that returns the area of a rectangle, then print the area of a 7 by 3 rectangle.
2. Write `count_word(text, word)` that returns how many times `word` appears in `text` as a whole word, ignoring capital letters, and test it on a sentence.
3. Write `percent_change(old, new)` that returns the change from `old` to `new` as a percentage, and use it on 1,200 and 1,500.

1.12 Libraries, import and help

A **library** (also called a **package**) is ready-made code that someone else has written. Before you use one in a notebook you **import** it, usually once, near the top. There are three common forms (Table 1.11).

Table 1.11. Three ways to import.

Form	How you then use it
`import math`	`math.sqrt(36)`

Form	How you then use it
`import pandas as pd`	`pd.read_csv(...)`; `as` gives the library a short nickname
`from math import sqrt`	`sqrt(36)`, without the library's name

By convention, pandas is imported as `pd`, NumPy as `np` and Matplotlib's charting module as `plt`, and you will see those nicknames in almost all Python code about data. A misspelled name stops with a `ModuleNotFoundError`, which also appears when a library is not installed:

```
import pandaz
```

```
ModuleNotFoundError: No module named 'pandaz'
```

To read the documentation of a function, pass it to `help()`, or type its name followed by a question mark, such as `len?`, which in Colab opens the help in a side panel. While you type, Colab also shows a short description of the function in a pop-up, and pressing `Ctrl` + `Space` lists the names that could come next.

```
help(round)
```

```
Help on built-in function round in module builtins:

round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.
    Otherwise the return value has the same type as the number.  ndigits
    may be negative.
```

`!pip install` lasts only until the session ends. Keep it in the first cell, so the notebook still runs tomorrow.

Python's standard installation does a great deal, but most of the tools journalists use come from libraries. Tables 1.12 to 1.14 list the ones you are most likely to meet, grouped by the job they do. The column "In Colab" says whether the library is already installed there; if it is not, the table gives the command to run in a code cell first. The exclamation mark at the start of `!pip install` tells Colab to pass the line to the computer's command line rather than to Python.

Table 1.12. Libraries for collecting data.

Library	What it does	In Colab	Import
Requests	Downloads web pages and data from APIs	Installed	`import requests`
Beautiful Soup	Reads the HTML of a web page so you can pull out text and tables	Installed	`from bs4 import BeautifulSoup`
Scrapy	Crawls many pages of a website in a structured way	Not installed; run `!pip install scrapy`	`import scrapy`
Tweepy	Collects posts through the X (formerly Twitter) API, which needs a developer account	Installed	`import tweepy`

Table 1.13. Libraries for handling and analyzing data.

Library	What it does	In Colab	Import
pandas	Tables: reading, cleaning, filtering, grouping, summarizing	Installed	`import pandas as pd`
NumPy	Fast calculations on large arrays of numbers; pandas is built on it	Installed	`import numpy as np`
Dask	Works with tables too large to fit in the computer's memory	Installed	`import dask.dataframe as dd`
sqlite3	Reads and writes SQLite databases; part of Python itself	Installed	`import sqlite3`
openpyxl	Reads and writes Excel files; pandas uses it behind the scenes	Installed	`import openpyxl`

Table 1.14. Libraries for charts and maps.

Library	What it does	In Colab	Import
Matplotlib	The basic charting library that most others build on	Installed	`import matplotlib.pyplot as plt`
seaborn	Statistical charts with less code, built on Matplotlib	Installed	`import seaborn as sns`

Library	What it does	In Colab	Import
Plotly	Interactive charts that readers can hover over and zoom	Installed	`import plotly.express as px`
Altair	Charts described as data, layer by layer	Installed	`import altair as alt`
GeoPandas	Maps and other geographic data	Installed	`import geopandas as gpd`

You will still find Newspaper3k, a library for extracting the text of news articles, in many tutorials. Its last release on PyPI, the official index of Python packages, came out on 28 September 2018 [7], so it is no longer a safe choice for new work.

> **Tip**
>
> **Which version do I have?** In Colab, `!pip show pandas` prints the installed version of pandas and some details about it, or a warning if it is not installed, and `!pip list` lists every installed package with its version. Inside Python, most libraries report their own version: after `import pandas as pd`, `pd.__version__` returns it. Compare it with the versions in the "As of October 2026" box in section 1.3 when your results differ from the book's.

```
import pandas as pd
pd.__version__
```

```
'2.2.3'
```

1.13 Errors you will meet

Everyone who writes code sees error messages every day. An error is not a disaster; it is Python telling you, as precisely as it can, what it could not do. The message can be long, but the most useful part is its last line, which names the type of error and gives a short explanation. Table 1.15 lists the errors you will meet most often in this book.

Table 1.15. Common errors and what to do about them.

Error	What it means	What to do
`SyntaxError`	The code is written incorrectly: a missing colon, bracket or quotation mark	Check the punctuation of the line shown

Error	What it means	What to do
`IndentationError`	The indentation is wrong	Indent four spaces under `if`, `for` and `def` lines
`NameError`	Python does not know the name	Check the spelling, or run the cell that creates the variable
`TypeError`	The operation does not work with that type of value	Convert the value, for example with `str()` or `int()`
`ValueError`	The type is right but the value is not, as in `int("hello")`	Clean the value first
`IndexError`	The position does not exist	The last position is the length minus 1
`KeyError`	The key is not in the dictionary, or the column is not in the table	Check the spelling and capitals, or use `.get()`
`ZeroDivisionError`	Division by zero	Check the number you divide by
`ModuleNotFoundError`	The library is not installed, or its name is misspelled	Fix the name, or install the library

In a notebook, a `NameError` very often means only that you reopened the notebook and ran a cell without first running the cell above it that created the variable. **Runtime > Run all** runs every cell from the top and fixes it.

Sometimes an error is expected and you want the code to carry on. A `try` block runs code that may fail, and an `except` block says what to do if it does:

```
values = ["1500", "1,200", "n/a", "980"]
for v in values:
    try:
        print(int(v))
    except ValueError:
        print("Could not read:", v)
```

```
1500
Could not read: 1,200
Could not read: n/a
980
```

Use `try` and `except` sparingly. An error that you silence is an error you no longer see, and in data work a value that could not be read is often the first sign that something in the data needs a closer look.

Your turn

1. Write code that causes a `NameError` on purpose, catch it with `try` and `except`, and print a message.
2. Inside `try` and `except`, try to read position 10 of `"spam"` and print the error message instead of stopping.
3. `total = "45"` is text. Convert it to a number, add 5 and print the result and its type.

1.14 DataFrames: your first table

Lists and dictionaries hold values; a **DataFrame** holds a whole table: rows of records and named columns, like a sheet in a spreadsheet. DataFrames come from pandas, the library that does most of the work in this book, imported under its usual nickname `pd`:

```
import pandas as pd

df = pd.DataFrame({
    "Title": ["Climate Change Report", "Election Results",
              "New Policy Updates"],
    "Author": ["Jane Doe", "John Smith", "Alice Brown"],
    "Views": [1500, 1200, 800],
    "Date": ["2024-01-01", "2024-01-02", "2024-01-03"]
})
df
```

	Title	Author	Views	Date
0	Climate Change Report	Jane Doe	1500	2024-01-01
1	Election Results	John Smith	1200	2024-01-02
2	New Policy Updates	Alice Brown	800	2024-01-03

We built the table from a dictionary: each key became a column name and each list became the values of that column. The names are invented, as they are in all the small practice tables of this chapter. Colab shows a DataFrame as a formatted table with three parts (Figure 1.10). The **index** down the left side labels the rows; pandas numbers them from 0 unless you say otherwise. The **column names** across the top say what each column holds. The **values** fill the cells, one for each combination of row and column. Each row is one record, here one article, and each column is one variable, here one fact about every article.

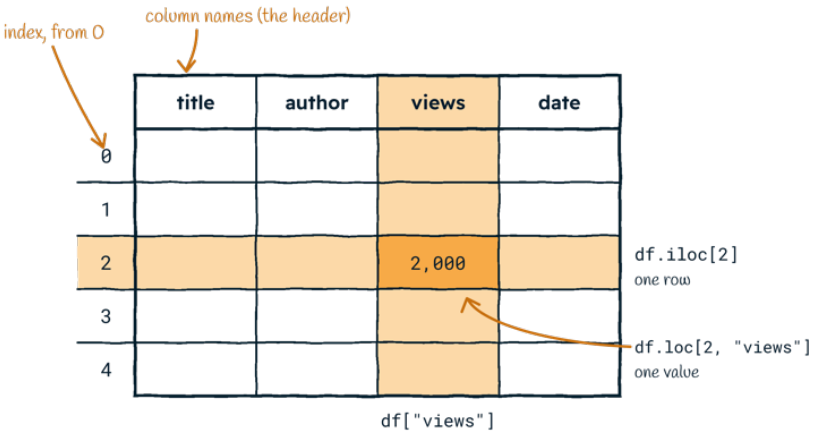


Figure 1.10. The parts of a DataFrame: the index labels the rows, the header names the columns, and each value sits where a row meets a column.

To add a column, assign a list of values to a new column name. The list must have one value for each row.

```
df["Category"] = ["Environment", "Politics", "Policy"]
df
```

	Title	Author	Views	Date	Category
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment
1	Election Results	John Smith	1200	2024-01-02	Politics
2	New Policy Updates	Alice Brown	800	2024-01-03	Policy

To add a row, the simplest method is `df.loc[len(df)]`. `len(df)` is the number of rows, 3, which is also the index of the next free row, and `.loc[]` places the new values there:

```
df.loc[len(df)] = ["Sports Update", "Bob White", 900, "2024-01-04", "Sports"]
df
```

	Title	Author	Views	Date	Category
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment
1	Election Results	John Smith	1200	2024-01-02	Politics
2	New Policy Updates	Alice Brown	800	2024-01-03	Policy
3	Sports Update	Bob White	900	2024-01-04	Sports

⚠ Common mistakes

You will often see older tutorials add rows with `df.append()`. That method was removed in pandas 2.0, and current versions of pandas stop with an error:

```
df.append({"Title": "Weather Alert"}, ignore_index=True)
```

```
AttributeError: 'DataFrame' object has no attribute 'append'
```

Use `df.loc[len(df)] = ...` as above, or `pd.concat()`, which Chapter 2 uses to join whole tables.

There are several ways to read from a DataFrame. Square brackets with a column name return that column; `.iloc[]` picks rows (and columns) by position, counting from 0; and `.loc[]` picks them by their labels.

```
df["Title"]
```

```
0    Climate Change Report
1          Election Results
2       New Policy Updates
3             Sports Update
Name: Title, dtype: object
```

A single column on its own is a **Series**, which Colab shows as plain text: the index, the values and a last line with the column's name and type. `.iloc[0]` returns the first row, also as a Series, with the column names down the left:

```
df.iloc[0]
```

```
Title       Climate Change Report
Author                   Jane Doe
Views                        1500
Date                   2024-01-01
Category              Environment
Name: 0, dtype: object
```

The row mixes text and numbers, so the Series falls back to `object`, the catch-all type.

`.loc[]` takes a row label and a column name, which here returns the author of the first article:

```
df.loc[0, "Author"]
```

```
'Jane Doe'
```

Column names must match exactly, capital letters included: `df.loc[0, "author"]` stops with a `KeyError`, because the column is called `Author`. You can also ask for several rows or several columns at once. A slice such as `0:2` takes the rows at positions 0 and 1, and a list of names inside the square brackets selects those columns:

```
df.iloc[0:2]
```

	Title	Author	Views	Date	Category
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment
1	Election Results	John Smith	1200	2024-01-02	Politics

```
df[["Title", "Views"]]
```

	Title	Views
0	Climate Change Report	1500
1	Election Results	1200
2	New Policy Updates	800
3	Sports Update	900

Notice the double square brackets in the last cell: the outer pair selects from the DataFrame, and the inner pair is the list of column names. Table 1.16 brings together the ways of adding and reading data in the structures of this chapter.

Table 1.16. Adding and reading data in lists, dictionaries and DataFrames.

Structure	Adding data	Reading data
List	`.append(item)`, `.extend([item1, item2])`	`my_list[0]` by position, `my_list[-1]` from the end
Dictionary	`my_dict["key"] = value`	`my_dict["key"]`, or `my_dict.get("key")` when the key may be missing
DataFrame	`df["new_column"] = [values]`, `df.loc[len(df)] = [values]`	`df["column"]`, `df.iloc[row]` by position, `df.loc[row, "column"]` by label

1.15 Hands-on exercises

The exercises in this section work with a small table of articles. Each one builds on the one before, so run them in order in the same notebook. Try each exercise yourself before you read the solution.

Exercise 1: Creating a DataFrame

Create a DataFrame called `df` from the data below, then show it.

Article	Views	Likes
Climate Report	1500	200
Election Update	2000	350
Tech Trends	1200	150

Solution

```
import pandas as pd

data = {
    "Article": ["Climate Report", "Election Update", "Tech Trends"],
    "Views": [1500, 2000, 1200],
    "Likes": [200, 350, 150]
}
df = pd.DataFrame(data)
df
```

	Article	Views	Likes
0	Climate Report	1500	200
1	Election Update	2000	350
2	Tech Trends	1200	150

`pd.DataFrame()` turns the dictionary into a table: each key becomes a column, and each list fills that column from top to bottom. A typing mistake in a column name now will cause errors later, so check the names when the table appears.

Exercise 2: Adding a column

Add a column called `Comments` with the values 50, 120 and 30, and show the table again.

Solution

```
df["Comments"] = [50, 120, 30]
df
```

	Article	Views	Likes	Comments
0	Climate Report	1500	200	50
1	Election Update	2000	350	120
2	Tech Trends	1200	150	30

The list must have exactly one value for each row. With two or four values, pandas would stop with a `ValueError` saying that the length of the values does not match the length of the index.

Exercise 3: Adding a row

Add a row for the article “Sports Highlights”, with 1,800 views, 250 likes and 80 comments.

Solution

```
df.loc[len(df)] = ["Sports Highlights", 1800, 250, 80]
df
```

	Article	Views	Likes	Comments
0	Climate Report	1500	200	50
1	Election Update	2000	350	120
2	Tech Trends	1200	150	30
3	Sports Highlights	1800	250	80

`len(df)` was 3 before the new row, so the row was added with index 3, after the last one. The values must be in the same order as the columns.

Exercise 4: Reading single values

Find the number of likes of the second article (Election Update), the number of comments of the last article and the number of views of “Tech Trends”.

Solution

```
print("Likes of the second article:", df.loc[1, "Likes"])
print("Comments of the last article:", df["Comments"].iloc[-1])

tech = df[df["Article"] == "Tech Trends"]
print("Views of Tech Trends:", tech["Views"].iloc[0])
```

```
Likes of the second article: 350
Comments of the last article: 80
Views of Tech Trends: 1200
```

`df.loc[1, "Likes"]` reads the row labeled 1, the second row, and the column `Likes`. `df["Comments"].iloc[-1]` takes the `Comments` column and then its last value. The third question is a search: `df["Article"] == "Tech Trends"` compares every title with “Tech Trends”, and putting that comparison inside `df[...]` keeps

only the rows where it is true. The result is still a table, even though it has one row, so `.iloc[0]` takes the first (and only) value of its `Views` column.

Exercise 5: Adding a calculated column

Add a column `Engagement Rate (%)` that holds likes plus comments as a percentage of views.

Solution

```
df["Engagement Rate (%)"] = (df["Likes"] + df["Comments"]) / df["Views"] * 100
df
```

	Article	Views	Likes	Comments	Engagement Rate (%)
0	Climate Report	1500	200	50	16.666667
1	Election Update	2000	350	120	23.500000
2	Tech Trends	1200	150	30	15.000000
3	Sports Highlights	1800	250	80	18.333333

The calculation works on whole columns at once: pandas adds the likes and comments of each row, divides by that row's views, and multiplies by 100. The parentheses make the addition happen before the division. Without them, Python would divide the comments by the views first and then add the likes.

Exercise 6: Filtering rows

Keep only the articles with 1,500 views or more, in a new DataFrame called `df_filtered`.

Solution

```
df_filtered = df[df["Views"] >= 1500]
df_filtered
```

	Article	Views	Likes	Comments	Engagement Rate (%)
0	Climate Report	1500	200	50	16.666667
1	Election Update	2000	350	120	23.500000
3	Sports Highlights	1800	250	80	18.333333

The comparison `df["Views"] >= 1500` produces `True` or `False` for every row, and `df[...]` keeps the rows marked `True`. Tech Trends, with 1,200 views, is left out. Notice that the remaining rows keep their original index labels (0, 1 and 3); the index identifies each row, so pandas does not renumber it after a filter. Because the

labels stay, `.loc[len(df)]` adds a row safely only to a table numbered 0, 1, 2 and so on: here `len(df_filtered)` is 3, a label that already exists, so a row placed at `df_filtered.loc[3]` would overwrite Sports Highlights.

A challenge that combines the steps

Create a new DataFrame of four articles with the columns `Article`, `Views`, `Likes` and `Comments`, add the engagement rate, and keep the articles with 300 likes or more.

Solution

```
df = pd.DataFrame({
    "Article": ["Article A", "Article B", "Article C", "Article D"],
    "Views": [1000, 2000, 1500, 1800],
    "Likes": [200, 400, 300, 250],
    "Comments": [50, 100, 70, 90]
})
df["Engagement Rate (%)"] = (df["Likes"] + df["Comments"]) / df["Views"] * 100
df_filtered = df[df["Likes"] >= 300]
df_filtered
```

	Article	Views	Likes	Comments	Engagement Rate (%)
1	Article B	2000	400	100	25.000000
2	Article C	1500	300	70	24.666667

Two articles have 300 likes or more. Article C is kept because `>=` includes the value 300 itself; with `>` it would have been left out.

1.16 Project: building and analyzing a table of articles

This project puts the chapter together. You will build a small table of articles, look at it in several ways, add columns, filter it and sort it. Open a new notebook (**File > New notebook in Drive**) and work through the steps in order.

Step 1: Import pandas

```
import pandas as pd
```

Step 2: Create the DataFrame

This time we start from a list of dictionaries, one dictionary for each article. It is a common shape for data that comes from a website or an API, and pandas turns it into a table directly:

```
articles = [
    {"title": "Climate Change Report", "author": "Jane Doe",
     "views": 1500, "date": "2024-01-01"},
    {"title": "Election Results", "author": "John Smith",
     "views": 1200, "date": "2024-01-02"},
    {"title": "Policy Update", "author": "Alice Brown",
     "views": 800, "date": "2024-01-03"},
    {"title": "Sports Highlights", "author": "Bob White",
     "views": 900, "date": "2024-01-04"},
    {"title": "Economic Analysis", "author": "Emily Davis",
     "views": 2000, "date": "2024-01-05"}
]

df = pd.DataFrame(articles)
df
```

	title	author	views	date
0	Climate Change Report	Jane Doe	1500	2024-01-01
1	Election Results	John Smith	1200	2024-01-02
2	Policy Update	Alice Brown	800	2024-01-03
3	Sports Highlights	Bob White	900	2024-01-04
4	Economic Analysis	Emily Davis	2000	2024-01-05

Step 3: Look at the table

Three commands give a first view of any table. `.head()` shows the first five rows, which matters little here but saves a great deal of scrolling when a table has thousands of rows; `.head(2)` shows two.

```
df.head(2)
```

	title	author	views	date
0	Climate Change Report	Jane Doe	1500	2024-01-01
1	Election Results	John Smith	1200	2024-01-02

`.dtypes` lists the type of each column:

```
df.dtypes
```

```
title     object
author    object
views      int64
date      object
dtype: object
```

`views` holds whole numbers (`int64`), and the other columns are marked `object`, which is how pandas 2.2 labels text. (pandas 3, released in January 2026, labels them `str`; you will see that if you run Python on your own computer, but Colab still uses pandas 2.2.3.) The dates are still text too: pandas does not recognize them as dates until we convert them, which Chapter 3 shows. `.describe()` summarizes the numeric columns:

```
df.describe()
```

	views
count	5.000000
mean	1280.000000
std	486.826458
min	800.000000
25%	900.000000
50%	1200.000000
75%	1500.000000
max	2000.000000

One viral story can drag the mean far above a typical article. The median barely moves, so check both.

The table counts the values (5) and gives their mean (1,280 views), their standard deviation (`std`, a measure of how spread out they are), the smallest and largest values, and the quartiles: 900, 1,200 and 1,500 are the values a quarter, half and three quarters of the way up the list of views sorted from smallest to largest. The middle one, 1,200, is the median. Chapter 4 explains these measures and when to use each one.

Step 4: Add columns

Add a category for each article, and a column that holds the average number of views:

```
df["category"] = ["Environment", "Politics", "Policy", "Sports", "Economics"]
df["average_views"] = df["views"].mean()
df
```

	title	author	views	date	category	average_views
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment	1280.0
1	Election Results	John Smith	1200	2024-01-02	Politics	1280.0
2	Policy Update	Alice Brown	800	2024-01-03	Policy	1280.0
3	Sports Highlights	Bob White	900	2024-01-04	Sports	1280.0
4	Economic Analysis	Emily Davis	2000	2024-01-05	Economics	1280.0

`df["views"].mean()` is a single number, 1,280, so pandas repeats it in every row. A column like this is useful mainly as a step toward something else, such as the difference between each article and the average, which you could add with `df["views"] - df["average_views"]`.

Step 5: Filter the rows

Keep the articles with more than 1,000 views:

```
popular_articles = df[df["views"] > 1000]
popular_articles
```

	title	author	views	date	category	average_views
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment	1280.0
1	Election Results	John Smith	1200	2024-01-02	Politics	1280.0
4	Economic Analysis	Emily Davis	2000	2024-01-05	Economics	1280.0

Or the articles in one category:

```
politics_articles = df[df["category"] == "Politics"]
politics_articles
```

	title	author	views	date	category	average_views
1	Election Results	John Smith	1200	2024-01-02	Politics	1280.0

Step 6: Sort the rows

`.sort_values()` sorts by a column; `ascending=False` puts the largest values first.

```
df_sorted = df.sort_values(by="views", ascending=False)
df_sorted
```

	title	author	views	date	category	average_views
4	Economic Analysis	Emily Davis	2000	2024-01-05	Economics	1280.0
0	Climate Change Report	Jane Doe	1500	2024-01-01	Environment	1280.0
1	Election Results	John Smith	1200	2024-01-02	Politics	1280.0
3	Sports Highlights	Bob White	900	2024-01-04	Sports	1280.0
2	Policy Update	Alice Brown	800	2024-01-03	Policy	1280.0

The most-read article, Economic Analysis, is now at the top. Sorting does not change `df` itself: it returns a sorted copy, which we stored as `df_sorted`.

Check your knowledge

1. What does `.head()` do?
   a. Shows the last rows of a DataFrame
   b. Shows the first rows of a DataFrame
   c. Deletes the first rows of a DataFrame
   d. None of the above

2. How do you add a new column to a DataFrame?
   a. `df.add_column("new_column", values)`
   b. `df["new_column"] = values`
   c. `df.insert("new_column", values)`
   d. `df.create_column("new_column", values)`

3. What does `.describe()` return?
   a. A list of column names
   b. Summary statistics for the numeric columns
   c. The unique values in each column
   d. A filtered DataFrame

4. Which line keeps the rows where `views` is greater than 1,000?
   a. `df[df["views"] > 1000]`
   b. `df.filter("views" > 1000)`
   c. `df.select("views" > 1000)`
   d. `df.views > 1000`

5. What does `.sort_values()` do?
   a. Deletes rows that meet a condition
   b. Sorts the DataFrame by one or more columns
   c. Adds a new column
   d. Filters rows that meet a condition

Answers: 1 b; 2 b; 3 b; 4 a (option d returns `True` or `False` for every row, but does not keep any rows; you need `df[...]` around it); 5 b.

1.17 Habits worth keeping

A few habits make notebooks easier to trust and easier to share. Write a text cell at the top that says what the notebook is for and where its data came from, and add a short comment wherever a line of code needs a reason. Give variables names that a colleague could understand without asking you. Work in small steps: run each cell as soon as you write it and look at the result before you write the next one, because an error is much easier to find in three new lines than in thirty. And before you share a notebook, choose **Runtime > Restart session and run all**. If every cell still runs from top to bottom in a fresh session, the notebook does not depend on something you typed and later deleted.

Chapter summary

- Google Colab runs Python in your browser, with no installation, and saves notebooks to Google Drive. A notebook is a list of code cells and text cells; `Shift` + `Enter` runs a cell, and a cell shows the value of its last line.
- A variable is a name for a value. The basic types are integers, floats, strings and Booleans; `type()` tells you which one you have, and `int()`, `float()` and `str()` convert between them.
- Strings have methods for cleaning and searching text (`.strip()`, `.lower()`, `.replace()`, `.split()`), and can be indexed and sliced like lists. f-strings put values into sentences.
- Lists keep items in order and can change; dictionaries store values under keys; tuples are fixed groups of values; sets keep only distinct values.
- `if`, `elif` and `else` make decisions; `for` loops repeat an action for each item. Python groups lines by their indentation, by convention four spaces per level.

- Functions, defined with `def`, package a calculation under a name; `return` hands back a result.
- Libraries are imported with `import`; `help()` shows their documentation. The last line of an error message names the problem.
- In a DataFrame, `df["column"]` selects a column, `.iloc[]` and `.loc[]` select by position and by label, `df[condition]` filters rows, `.sort_values()` sorts them, and `.head()`, `.dtypes` and `.describe()` give a first view of a table.

Solutions to the “Your turn” exercises

The solutions below are one way to answer each exercise; there are often others. Each cell runs on its own.

Section 1.5: Variables and types

```
# 1
price = 20
price += 5
print(price, type(price))

# 2
full_name = "Amr Eleraqi"
age = 40
city = "Toronto"
print(full_name, age, city)

# 3
second_place = "Beirut"
my_name = "Sara"
class_name = "Data journalism"
print(second_place, my_name, class_name)

# 4
answer = "42"
print(int(answer) + 8)

# 5
print(int(9.99))   # the .99 is cut off, not rounded
```

```
25 <class 'int'>
Amr Eleraqi 40 Toronto
Beirut Sara Data journalism
50
9
```

Section 1.6: Numbers and arithmetic

```python
# 1
area = 7.5 * 4
print(area, round(area, 1))

# 2
print(24 * 60 * 60)

# 3
print(2026 % 2 == 0)

# 4
print(round((18 + 25 + 31) / 3, 2))

# 5
print(abs(-4 - 11))
```

```
30.0 30.0
86400
True
24.67
15
```

Section 1.7: Comparisons and logic

```python
# 1
age = 20
print(age >= 20 and age < 30)

# 2
password = "secret12"
print(len(password) >= 8)

# 3
headline = "Nile water levels drop"
print("water" in headline)

# 4
views_yesterday = 900
views_today = 1200
print("Difference:", views_today - views_yesterday)
print("Total:", views_today + views_yesterday)
if views_today > views_yesterday:
    print("Views went up.")
```

```
elif views_today < views_yesterday:
    print("Views went down.")
else:
    print("Views did not change.")
```

```
True
True
True
Difference: 300
Total: 2100
Views went up.
```

Section 1.8: Working with text

```
# 1
email = "  Amr.Eleraqi@Example.COM  "
print(email.strip().lower())

# 2
words = "data journalism is fun".split()
print(len(words))

# 3
print("the dam and the river and the data".split().count("the"))

# 4
word = "journalism"
print(word[:4], word[-3:], word[::-1])

# 5
phone = "416-555-0199"
print(phone[:3], phone[-4:])

# 6
city = "Cairo"
population = 10230350
print(f"{city} has {population:,} people.")
```

```
amr.eleraqi@example.com
4
3
jour ism msilanruoj
416 0199
Cairo has 10,230,350 people.
```

In exercise 3, splitting the text into words before counting matters: `.count("the")` on the whole text would also count “the” inside longer words such as “there” or “other”, while `.split().count("the")` counts only the word itself.

Section 1.9: Lists, dictionaries, tuples and sets

```
# 1
scores = [70, 95, 88]
scores.append(60)
scores.remove(70)
scores.sort()
print(scores)

# 2
cities = ["Cairo", "Toronto", "Beirut", "Tunis", "Amman"]
print(len(cities), cities[2])
cities.insert(1, "Alexandria")
print(cities)

# 3
tags = ["data", "water", "data", "nile"]
print(tags.count("data"), len(set(tags)))

# 4
article = {"title": "Nile water", "year": 2026, "tags": ["water", "Egypt"]}
print(article["year"], article["tags"][1])

# 5
article["author"] = "Amr Eleraqi"
article["year"] = 2027
print(article)

# 6
data = [1, 2, [3, 4, ["deep"]]]
print(data[2][2][0])
```

```
[60, 88, 95]
5 Beirut
['Cairo', 'Alexandria', 'Toronto', 'Beirut', 'Tunis', 'Amman']
2 3
2026 Egypt
{'title': 'Nile water', 'year': 2027, 'tags': ['water', 'Egypt'], 'author': 'Amr
Eleraqi'}
deep
```

Section 1.10: Decisions and repetition

```
# 1
score = 85
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
```

```
    print("C")
else:
    print("F")

# 2
for n in range(2, 11, 2):
    print(n, end=" ")
print()

# 3
marks = [55, 78, 91, 64, 83]
for m in marks:
    if m > 70:
        print(m)

# 4
total = 0
for m in marks:
    total += m
print(total, total / len(marks))

# 5
headlines = ["Nile WATER levels drop", "Budget passes",
             "Water prices rise", "Election results"]
count = 0
for h in headlines:
    if "water" in h.lower():
        count += 1
print(count)
```

```
B
2 4 6 8 10
78
91
83
371 74.2
2
```

Section 1.11: Functions

```
# 1
def area(width, height):
    return width * height

print(area(7, 3))

# 2
def count_word(text, word):
    return text.lower().split().count(word.lower())
```

```
print(count_word("The Dog saw the other dog", "dog"))

# 3
def percent_change(old, new):
    return (new - old) / old * 100

print(percent_change(1200, 1500))
```

```
21
2
25.0
```

Section 1.13: Errors you will meet

```
# 1
try:
    print(undefined_name)
except NameError as e:
    print("NameError:", e)

# 2
try:
    print("spam"[10])
except IndexError as e:
    print("IndexError:", e)

# 3
total = "45"
total = int(total) + 5
print(total, type(total))
```

```
NameError: name 'undefined_name' is not defined
IndexError: string index out of range
50 <class 'int'>
```

`except NameError as e` stores the error under the name `e`, so the message Python would have shown can be printed instead.

Sources for this chapter

1. Python Software Foundation. “History and License,” **Python 3 documentation**. docs.python.org/3/license.html
2. Python Software Foundation. “Using Python on Windows,” **Python 3 documentation**. docs.python.org/3/using/windows.html
3. Python Software Foundation. “Python Releases for Windows.” python.org/downloads/windows
4. Google. “Colaboratory Release Notes,” entry of 16 September 2026. colab.research.google.com/notebooks/relnotes.ipynb

5. Google. “backend-info: pip-freeze.txt” (list of the Python packages installed in the Colab runtime), October 2026. github.com/googlecolab/backend-info
6. van Rossum, G., Warsaw, B., and Coghlan, A. **PEP 8: Style Guide for Python Code**. peps.python.org/pep-0008
7. “newspaper3k 0.2.8,” **Python Package Index**. pypi.org/project/newspaper3k

That was Chapter 1

The book goes on from here to the work a data story needs. Chapter 2 opens the files sources send you and collects data from Google Drive, Kaggle, GitHub, web pages and news feeds. Chapter 3 cleans a public register of 150,482 electric vehicles and finds out what its zeros really mean. Chapter 4 explores a table with summaries, charts and correlations before you trust any number in it. Each chapter ends with exercises and their solutions, a hands-on project or case study, and a quiz.

GET THE BOOK

Paperback · ISBN 9798309728350 · www.amazon.com/dp/B0DWKKD5F3

Hardcover · ISBN 9798309733132 · www.amazon.com/dp/B0DWG2DRKZ

Kindle edition · www.amazon.com/dp/B0DY6L9Q6T

NOTEBOOKS AND DATA

github.com/aeleraqi/Python-for-Journalists

Part of the Data + Stories series. This sample is free to read and to share as it is, without changes. Copyright © 2026 Amr Eleraqi. All rights reserved.